Advanced JavaScript Nightmare Assignment

Build a mini JavaScript utility library from scratch.
This assignment focuses on:

- Functions
- Closures
- Loops
- Problem Solving
- Weird JavaScript behavior
- Async flow
- Recursion

Rules:

- Do NOT use: map, filter, reduce, find, some, every, flat, flatMap, sort, includes, indexOf
- Allowed: loops, recursion, functions, arrays, objects, closures
- All implementations must handle edge cases.
- Avoid mutating original data unless required.

1. customMap

Create your own implementation of map.

```
const nums = [1,2,3];

const result = customMap(nums, function(x){
   return x * 2;
});

// [2,4,6]
```

2. customFilter

Create your own implementation of filter.

```
const users = [
 {name:"Ahmed", age:22},
 {name:"Sara", age:15}
];

// return adults only
```

3. customReduce

Create your own implementation of reduce.

```
const total = customReduce(
 [1,2,3],
 function(acc, current){
   return acc + current;
 },
 0
);
```

4. groupBy

Group array items based on a specific property.

```
groupBy(users, "role");
```

5. deepClone

Create a deep clone utility for objects and arrays.

```
const copy = deepClone(original);
```

6. once

Function can execute only once.

```
const init = once(fn);

init();
init();
```

7. memoize

Cache function results for performance optimization.

```
const fib = memoize(function(n){
   ...
});
```

8. compose

Compose multiple functions together.

```
compose(fn1, fn2, fn3);
```

9. flattenArray

Flatten deeply nested arrays.

```
[1,[2,[3,[4]]]]
```

10. createCounter

Build counter utility using closures only.

```
const counter = createCounter(10);
```

11. createSecretHolder

Protect private data using closures.

```
const secret = createSecretHolder("123");
```

12. pipeAsync

Create async pipeline utility.

```
pipeAsync(fn1, fn2, fn3);
```

13. Weird JavaScript

Explain and demonstrate weird JavaScript behaviors.

```
- this
- hoisting
- closures in loops
- bind/call/apply
```

14. Final Boss - Event Emitter

Build a small event emitter system.

```
emitter.on("login", callback);

emitter.emit("login", data);
```

Submission Requirements:

- GitHub Repository

- PROBLEMS.md file
- 5 minute explanation video

Evaluation:

- Clean Code
- Edge Cases
- Problem Solving
- Closures Understanding
- Weird JavaScript Understanding